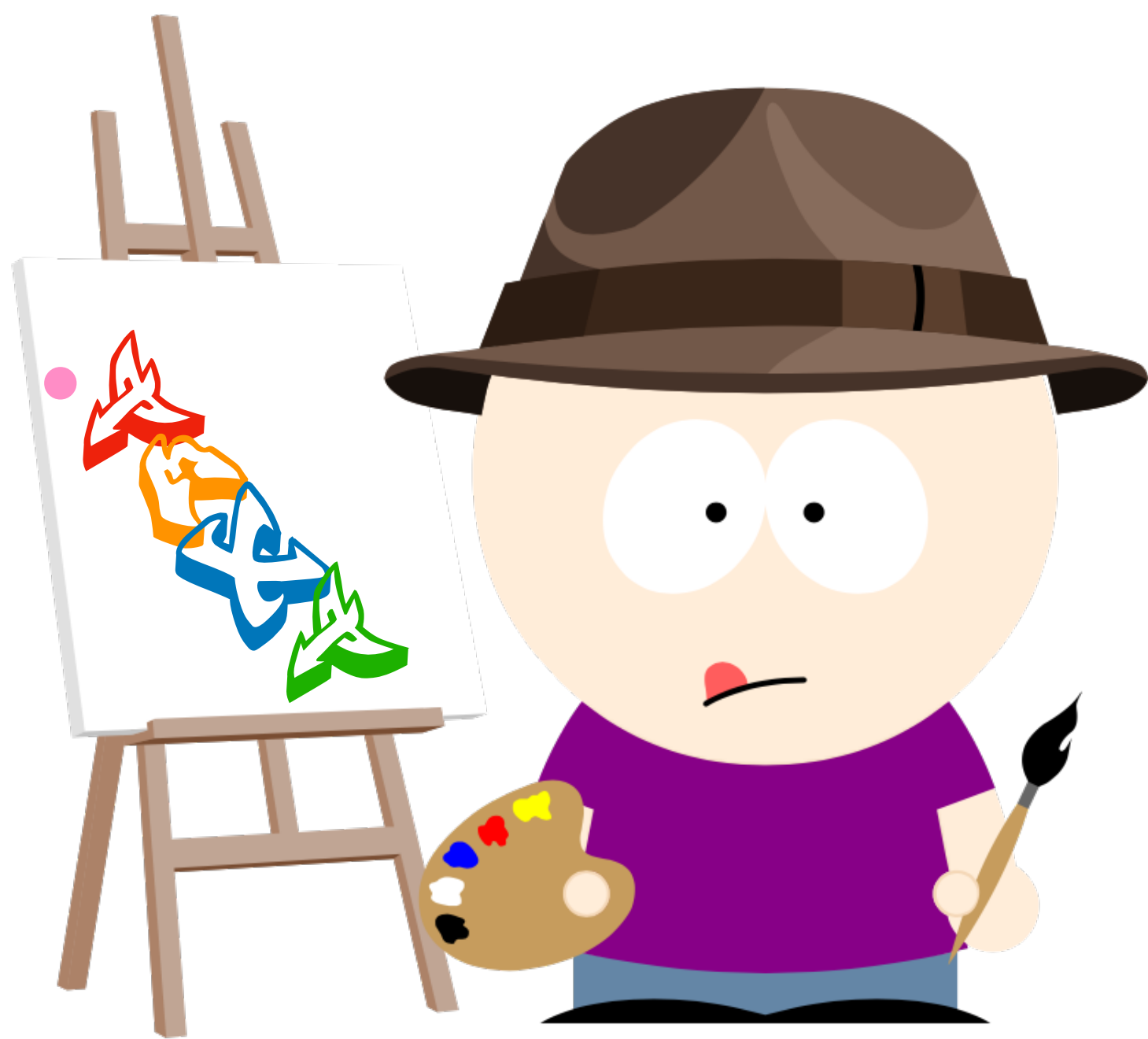




IN-CLASS EXERCISES



CODE GENERATION 2

Download exercises

1. Go to

<https://ligerlabs.org/compilers.html>

2. Download the file

`codegen-2-exercises.zip`

3. Open up a terminal and Unzip the file

```
> unzip codegen-2-exercises.zip
```

```
> cd codegen-2-exercises
```

```
> ls
```

Task 1

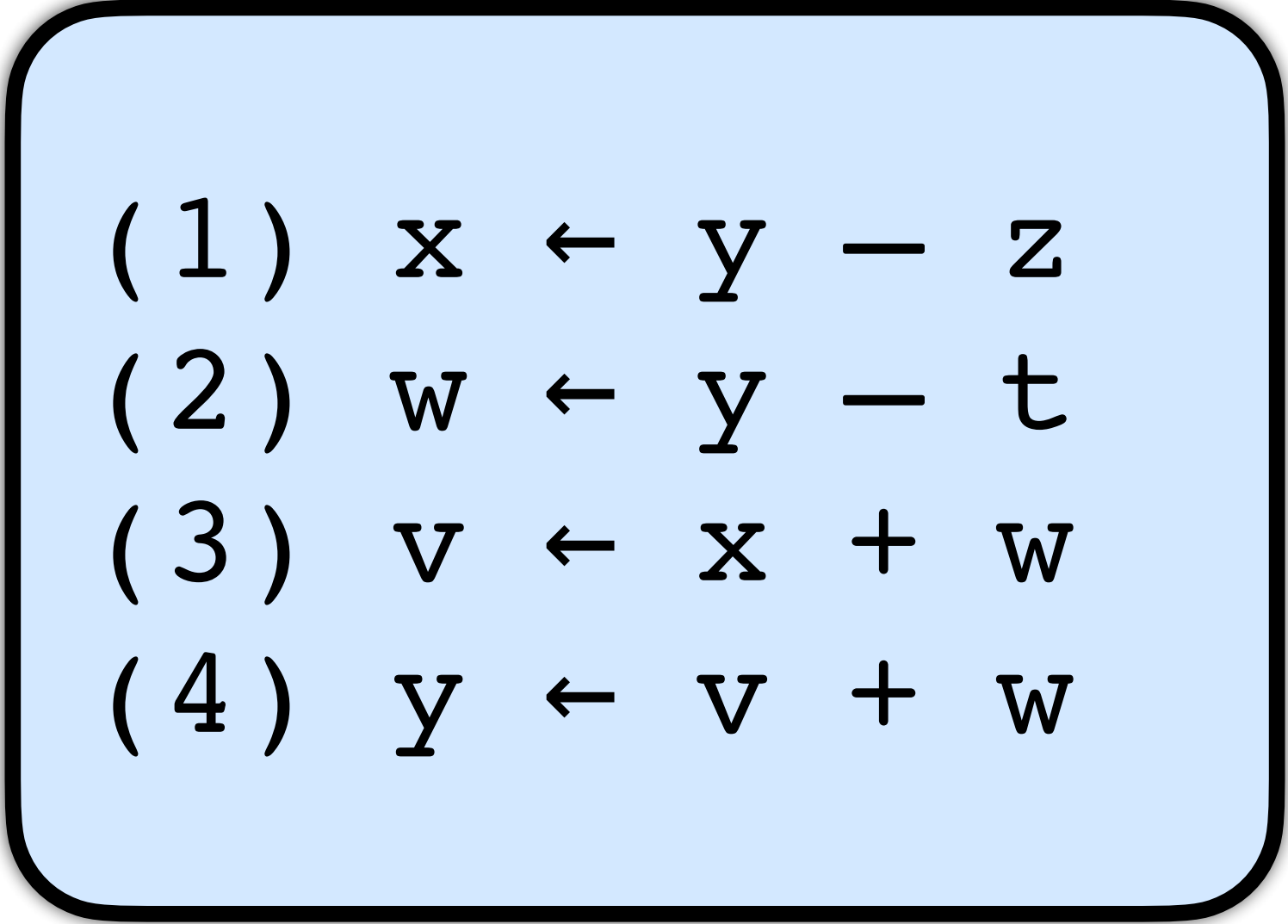
- **Without using the algorithm**, show the next use/liveness information for this basic block:

(1) $x \leftarrow y - z$
(2) $w \leftarrow y - t$
(3) $v \leftarrow x + w$
(4) $y \leftarrow v + w$

Live/Dead						
	x	y	z	w	t	v
(1)						
(2)						
(3)						
(4)						

Task 2

- **Using the next use/liveness algorithm**, show the steps the algorithm goes through on this basic block:



```
(1) x ← y - z  
(2) w ← y - t  
(3) v ← x + w  
(4) y ← v + w
```

Task 3

- Show the steps of the code generation algorithm on the basic block below.
- Liveness information has been provided for you.
- Make sure to show the changes to the address and register descriptors.

```
(1) X ← Y + 5  
(2) print X  
(3) Z ← Y * 7
```

Live/Dead			
	X	Y	Z
(1)	L	L	D
(2)	D	L	D
(3)	L	L	L

AD		
Var	Mem	Regs
X	0x0	
Y	0x4	
Z	0x8	

RD	
Reg	Cont
R0	
R1	

Task 4 (a)

- Show the steps of the code generation algorithm on the basic block below. Use the machine model discussed in class.
- Liveness information has been provided for you.
- Make sure to show the changes to the address and register descriptors.

```
(1) X ← Y * Z
(2) Y ← X + Z
(3) Z ← X * 7
(4) print X+Z
```

Live/Dead			
	X	Y	Z
(1)	L	D	L
(2)	L	D	D
(3)	L	D	L
(4)	L	L	L

AD		
Var	Mem	Regs
X	0x0	
Y	0x4	
Z	0x8	

RD	
Reg	Cont
R0	
R1	

Task 4 (b)

- Do the same as in task 4 (a), but use a MIPS machine model.



DEFINITIONS AND ALGORITHMS

1. Scan **forwards** over the basic block:
 - Initialize the symbol table entry for each used variable, and the tuple data for each tuple.
2. Scan **backwards** over the basic block.
for every tuple **(i): $X \leftarrow Y \text{ op } Z$** **do**
 1. Copy the live/next use-info from X, Y, Z 's symbol table entries into the tuple data for tuple (i).
 2. Update X, Y, Z 's symbol table entries:
 $X.\text{live} \leftarrow \text{DEAD};$
 $X.\text{next_use} \leftarrow \text{X};$
 $Y.\text{live} \leftarrow Z.\text{live} \leftarrow \text{LIVE};$
 $Y.\text{next_use} \leftarrow Z.\text{next_use} \leftarrow (i);$

Machine Model

Format	Address Computation
LOAD M, R	Load variable M into Register R
STORE R, M	Store register R into variable M
ADD M, R	Compute $R \leftarrow R + M$. Also SUB, MUL, DIV.
ADD R2, R1	Compute $R1 \leftarrow R1 + R2$. Also SUB, MUL, DIV.
MOV R2, R1	$R1 \leftarrow R2$

- The machine has n general purpose registers $R0, R1, \dots, Rn$.

GenCode(i: $X \leftarrow Y + Z$)

1. Find location L (often a register) in which to store x :

$L := \text{GetReg}(i: X \leftarrow Y + Z).$

2. Find Y' , the most favorable location for Y :

```
 $Y' := \text{if } (Y \text{ is in a register } R) \{$   
     $R$   
 $\} \text{ else } \{$   
     $Y$ 's memory location;  
 $\};$   
 $\text{if } (Y' \text{ is not in } L) \{$   
     $\text{gen}(\mathbf{LOAD} \ Y', L).$   
 $\}$ 
```


4. Find Z' , the best location for Z' :

```
     $Z' := \text{if } (Z \text{ is in a register } R) \{$   
         $R$   
     $\} \text{ else } \{$   
         $Z'$ 's memory location;  
     $\}$ 
```

5. Generate the add instruction:

```
    gen(ADD  $Z'$ ,  $L$ )
```

6. Update the **address descriptor**:

X is now in location L .

7. Update the **register descriptor**:

X is now only in location L ;

```
    if ( $Y$  has no next use) {Set "Y is not in any register."}
```

```
    if ( $Z$  has no next use) {Set "Z is not in any register."}
```


GenCode(i: $X \leftarrow Y$)

1. if (Y only in memory location L) {
 $R := \text{GetReg}()$; $\text{gen}(\mathbf{LOAD} \ Y, R)$;
 Address Descriptor: Y is now only in reg R .
 Register Descriptor: R now holds Y .
}
2. if (Y is in register R) {
 Address Descriptor: X is now only in register R .
 Register Descriptor: R now holds X .
 if(Y has no next use) {**RD:** No register holds Y .}
}
3. At the end of the basic block store all live variables (that are left in registers) in their memory locations.

GenReg(i: $X \leftarrow Y + Z$)

CASE 1: (Y is in register R) &&
(R holds only Y) &&
(Y has no next use)
 return R

CASE 2: there is an empty register R available
 return R;

CASE 3: (X has a next use) &&
(there is an an occupied register R)
 gen(**STORE** R, R's memory location);
 return R;

CASE 4:
 return the memory location of X.

COLLBERG.CS.ARIZONA.EDU

LIGERLABS.ORG

SUPPORTED BY
NSF SATC/TTP-1525820
SATC/EDU-2029632

COPYRIGHT © 2024-2026

CHRISTIAN COLLBERG

UNIVERSITY OF ARIZONA